



I need to improve the runtime of the attached MIQCP model without altering any of its functionality or features. What techniques can I use to achieve this, such as reformulating the constraints?

Read through the whole builder. One important finding first: despite the "MIQCP" label, the attached model is actually a **pure MILP** in its current form — there are no quadratic constraints left in the code (addLConstr everywhere, zero addQConstr/quadratic objective terms). The quadratic part was already eliminated once: the SoC-dependent aging weight $w(x) = 1 + 4 \cdot f \cdot (x - 0.5)^2$ is approximated by 9 piecewise-linear tangent lines precisely because "carrying the square itself made the model quadratically constrained and cost far more solver time than the shape is worth". That's exactly the right strategy, and it's the template for everything else below. ^[1]

Tighten the big-M values

Several indicator-style constraints use global big-M constants where much tighter, locally computable values exist. Tighter M directly strengthens the LP relaxation, which is where time-indexed scheduling models usually lose their runtime. ^[1]

- Chain timing (line 2126): `big_m_time = last_step + max(duration) + 2` is a global horizon (~50 steps). Per compatible pair (f, g) , the tightest valid value is $M_{fg} = \max S_g - \min S_f - (\text{dur}_f + \text{link}_{fg})$ from the sampled start lists — typically 2-4× smaller, and it lets presolve kill chain variables whose windows can't overlap.
- Driver-hour counter (lines 2395-2399): `big_m_steps = horizon + 2` can shrink per step t to the number of steps *remaining until the latest plausible depot return*, since the bound is only ever approached near the end of the shift anyway.
- SoC-at-departure (line 2472): `Mbig = trip_energy` is already the tight value (the constraint collapses to `soc ≥ trip_energy · z`). You can additionally replace this whole family with `addGenConstrIndicator(z[m,f,s]=1 → soc ≥ energy)` — Gurobi then branches on the binary directly instead of embedding a disjunctive term in every node LP. Same feasible set, better node bounds.

Reduce the variable families

The $z_{m,f,s}$ set is the combinatorial core, and every derived family (route_start, route_end, chain, break_active, penalized_external, `y_m ≥ z links`) scales with it. ^[1]

- Prune z by precomputation, not by the solver: drop start times s that can't be served given (a) the latest arrival back at the depot within work hours, (b) the initial SoC — a trip whose

energy exceeds what the truck can reach by charging from 00:00 to s can never start at s without a prior charging-free window. Time-expanded reachability from the day start prunes candidates *before* variables exist.

- Merge $E_{\text{pos}}/E_{\text{private}}/E_{\text{public}}$: with $E_{\text{private}} + E_{\text{public}} = E_{\text{pos}}$ and both capped (lines 2614–2616), one family is redundant — substitute $E_{\text{public}} = E_{\text{pos}} - E_{\text{private}}$ into the objective and station caps and drop it. Same model, $\sim|BEV|\times 48$ fewer variables and equality rows.
- Aggregate the $y_m \geq z[m, f, s]$ links (line 2491–2494): keep one superset link per (m, f) ($y_m \geq \sum_s z[m, f, s]$ is *invalid* here, but $y_m \geq z$ over real trips only can collapse into $|V|\times|F|$ rows added via listing: since each trip is assigned exactly once, single-row-per-trip form applies the same fixation logic with far fewer nonzeros in the LP).
- If your Gurobi version supports it, express the charging-curve derating (the `chg_full` binary + $E_{\text{public}} \geq \text{ext_minimum}$ block at 2657–2664) via `addGenConstrPWL` — the presolved linearization is tighter than the hand-rolled indicator pair and cheaper to maintain.

Break the vehicle symmetry

Trucks of the same type are interchangeable, and branch-and-bound enumerates their permutations — with copies sized by `fleet_pool_copies_for(day)` this can be the dominant term in the search tree. Options that keep the model functionally identical:^[1]

- Aggregate to type-level integers: $z_{k,f,s} \in \mathbb{Z}_{\geq 0}$ per vehicle *type* k instead of per truck, with capacity $\sum z_{k,f,s} \leq \text{copies}_k$ and per-type aggregate SoC/energy tracks. Any aggregate solution decomposes back into per-truck schedules in postprocessing (first-fit by time order). Valid as long as no constraint distinguishes identical trucks — which holds for the fleet pool; it needs care only where per-vehicle reportage matters (the EFC tracking at 2889–2909 is per truck).
- Cheaper first step: add static symmetry-breaking inequalities on the continuous energy state, e.g. $x_{\text{SoC},k}(t = 0\text{-aggregated}) \geq x_{\text{SoC},k+1}(\cdot)$ -style ordering for identical trucks, or order trucks by cumulative assignment $\sum_{f,s} z$. This doesn't change the set of optimal *schedules*, only removes permutation-equivalent ones. `penalty_vehicle_id_order` already biases this in the objective but doesn't constrain it.
- Also enable the fleet-day cap `pool_copies = fleet_pool_copies_for(...)` for disposition runs (currently used only in the design run at line 3198; the day run takes the roster verbatim from `fleet_input_file`). Spare trucks are provably unused beyond the headroom the sizer already adds — but note the caveat in section 3.3.10 of your code: V2G-only standby trucks don't count as "used", so check the headroom setting covers them if you rely on that behavior.

Parameter and workflow levers

- `MIPFocus=1` during parameter studies: your objective mixes ~ 0.15 €/kWh energy terms with 500 € rule-breach penalties, so finding *any* good incumbent early dominates; switch back to 0 for publication runs.

- Give branching priorities (`Var.BranchPriority`): high for z (assignment/timing), medium for chain/route variables, low for the energy/degradation continuous binaries. The energy layer is largely determined once the schedule is fixed.
- Coefficient scaling: the objective spans about 4 orders of magnitude (0.08–500 €). Set `ObjectiveScale` and/or `NumericFocus=1` — time-indexed models like this are known-degenerate (your own `DegenMoves` comment says so), and poor scaling worsens node LP solves.
- Un-comment `model.tune()` once on a representative instance and store the resulting `.prm` — that's cheaper than hand-tuning `Cuts/Presolve/Aggregate` and these settings carry across identical data shapes.
- Two-stage warm start is implemented but gated on `fleet_operation_mode == 'crewed'`. The same relaxed-solve → MIP start pattern helps every configuration: solve once with crew rules/Charging-curve binaries relaxed, hand `Variable.Start` values to the full model, keep the same seed (`monte_carlo_seed` is fixed, so the feasible set is reproducible).^[1]
- Build-time is also worth attacking: the loops at 2092–2101 and 2995–2996 scan full dictionaries per (m,f) — pre-index by trip (a `defaultdict` of `route_start/chain` per (m,f)) or use `tupledict.select()`. On large day sets model *construction* can exceed the solve time.

Structural options (bigger surgery)

If the above plateau, the assignment-plus-timing structure is a natural candidate for **Dantzig-Wolfe decomposition**: a set-partitioning master over precomputed feasible daily "routes" per vehicle (trip chains + SoC-valid charging/V2G stops), with a shortest-path-with-resources pricing subproblem generating columns. The depot-wide coupling constraints (site peak, charger capacity, V2V pairing) stay in the master as coupling rows. This usually buys an order of magnitude on instances where the compact formulation's LP bound is weak, at the cost of a real implementation investment.

Priority order to try

1. Per-pair big-M tightening + variable pruning (cheap, often the largest single win on weak-relaxation models like this one)
2. Fleet-copy capping + symmetry handling (tree size)
3. Variable elimination (`E_public` family) and `indicator→GenConstr` conversions (node LP size)
4. Tuner run + branching priorities + warm start for all modes (solver behavior)
5. Column generation (only if 1–4 don't suffice)

The file is fully commented with rationale per formulation choice (e.g., the PWL aging tangents at section 3.4), so any reformulation is worth documenting the same way — "changed the model without changing its function" is exactly the class of change that silently alters study outcomes if the reasoning isn't written down.^[1]

1. hdv_disposition_optimization.py